



Introdução

O FreeRTOS consiste em um kernel de sistema operacional de tempo real, com código-fonte aberto e distribuição gratuita, para dispositivos microcontrolados. A documentação está disponível em https://www.freertos.org. O FreeRTOS foi projetado para ser pequeno e simples. Ele é escrito, em quase sua totalidade, na linguagem de programação C facilitando a portabilidade e a manutenção. Atualmente está portada para 35 modelos de microcontroladores. Também apresenta algumas funções escritas na linguagem Assembly, especialmente nas rotinas de agendamento de tarefas que habitualmente são específicas para cada arquitetura.

Os Sistemas Operacionais de Tempo Real são implementados de maneira similar aos sistemas de tempo compartilhado, sendo que a principal diferença entre eles é o tempo exigido no processamento das aplicações. Enquanto nos sistemas de tempo compartilhado o tempo de processamento pode variar sem comprometer as aplicações em execução, nos sistemas de tempo real os tempos de processamento devem estar dentro de limites rígidos.

Desta maneira, não existe o conceito de time slice (fatia de tempo), então um programa utiliza o processador o tempo que for necessário ou até que apareça um outro mais prioritário. Assim sendo, quem define a prioridade é o próprio programa e não o sistema operacional.

Estão presentes em aplicações críticas, por exemplo, monitoramento de refinarias de petróleo, veículos, tráfego aéreo, usinas termoelétricas e nucleares.

Biblioteca para Arduino

Instalação da biblioteca FreeRTOS no ambiente de desenvolvimento (IDE) do Arduino:

1) Escolher no menu “Tools” a opção “Manage Libraries...”.
2) Procurar pela biblioteca FreeRTOS (Figura 1).
3) Selecionar e versão mais recente e clicar no botão “Install”.

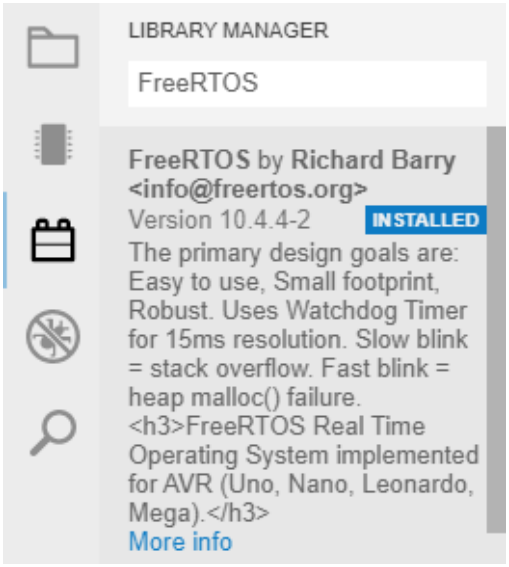


Figura 1: Instalação da Biblioteca FreeRTOS

Circuito Eletrônico

A Figura 2 mostra a montagem do circuito eletrônico que será utilizado nos programas que serão desenvolvidos:

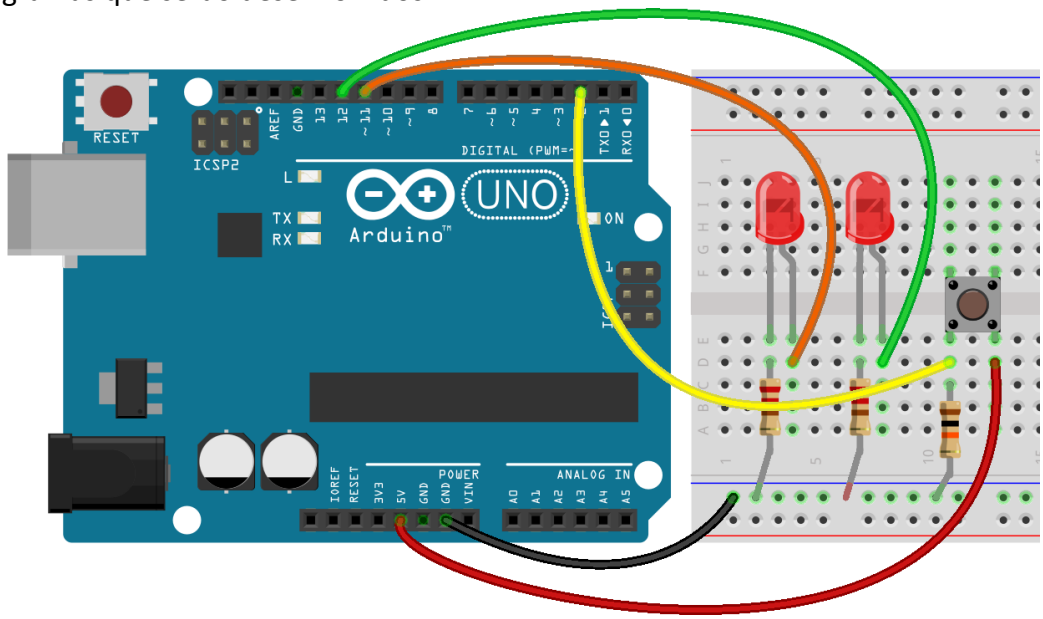


Figura 2: Circuito Eletrônico

Lista de Materiais:

- 1 Arduino Uno R3 ou similar.
- 2 Resistores de 220 Ohms (vermelho, vermelho, marrom) ou 330 Ohms (laranja, laranja, marrom).
- 2 LEDs (qualquer cor).
- 1 Chave Táctil.
- 1 Resistor de 10k Ohms.
- 1 Protoboard.
- Jumper cable.

Simulador Wokwi

Os programas também poderão ser implementados no Simulador Wokwi (Figura 3), pois este apresenta suporte ao uso da biblioteca do FreeRTOS entre muitas outras. O simulador não necessita de instalação, sendo acessado diretamente pelo navegador, através do endereço https://wokwi.com.

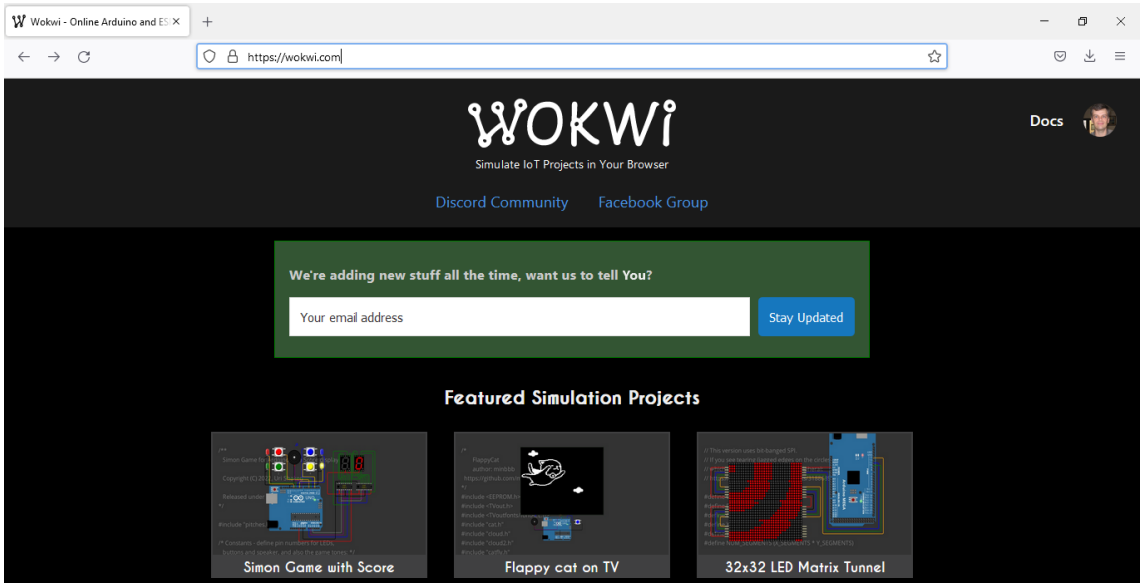


Figura 3: Simulador Wokwi

Na Figura 4 é possível observar o circuito, que será usado nos programas que serão desenvolvidos, devidamente implementado no Wokwi.

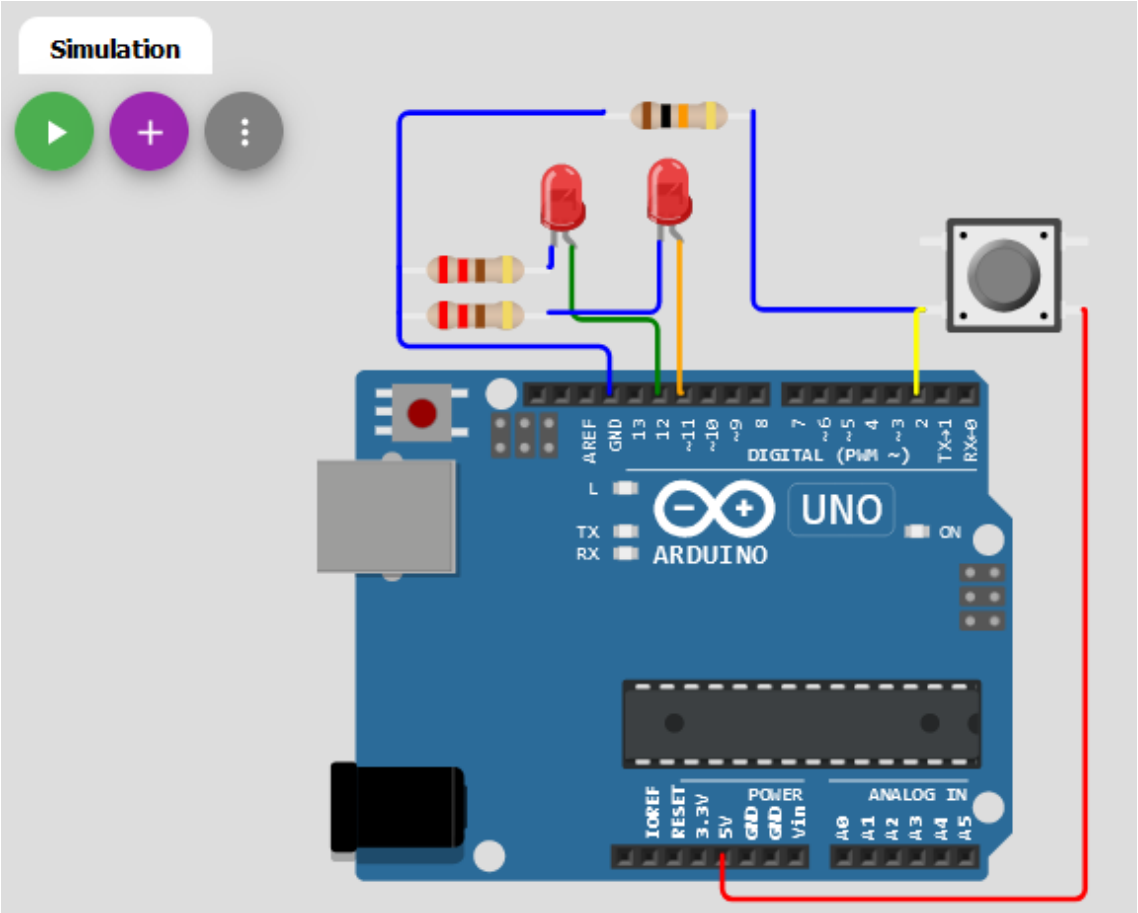


Figura 4: Circuito Implementado no Wokwi

Também é necessário acessar a aba Library Manager e adicionar a biblioteca do FreeRTOS, conforme ilustra a Figura 5.

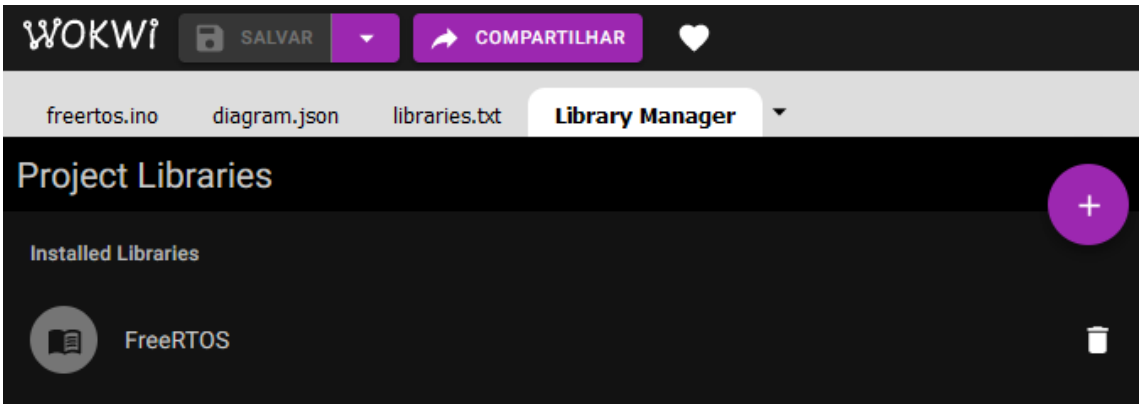


Figura 5: Aba do Gerenciador de Bibliotecas do Wokwi

Tarefas

Em linhas gerais, podemos entender que tarefas consistem em uma sequência de instruções (programas), escalonadas e gerenciadas pelo sistema operacional, podendo ser executadas por um ou mais processadores, dependendo da arquitetura utilizada.

O programa a seguir demonstra a criação de tarefas em um programa para o Arduino usando o FreeRTOS. Note que a funcionalidade do programa é implementada através de uma tarefa ao invés de ser desenvolvida na função loop do programa. Desta maneira, na função setup, realizamos a criação da tarefa e a inicialização do gerenciador de tarefas.

```
#include <Arduino_FreeRTOS.h>

void tarefaLed(void *parametros) {
  pinMode(13, OUTPUT);
  while(1) {
    digitalWrite(13, HIGH);
    vTaskDelay(500 / portTICK_PERIOD_MS);
    digitalWrite(13, LOW);
    vTaskDelay(500 / portTICK_PERIOD_MS);
  }
}

void setup() {
  xTaskCreate(tarefaLed, "LED", 128, NULL, 1, NULL);
  vTaskStartScheduler();
}

void loop() {
}
```

A função xTaskCreate será responsável pela criação da tarefa, sendo que devemos passar como parâmetros:

- pvTaskCode: Ponteiro para a função que contém a tarefa a ser criada. Esta função não deve ter retorno, isto é, deve ser declarada como void.
- pcName: String contendo o nome da tarefa.
- usStackDepth: Número de palavras (words) de memória a serem alocadas como stack da tarefa.
- pvParameters: Parâmetros que devem ser enviados para a tarefa.
- uxPriority: Prioridade da tarefa.
- pxCreatedTask: Handle para a tarefa. Este parâmetro apenas deve ser especificado quando é necessário suspender e reiniciar a tarefa durante a execução do programa.

Em seguida, a função vTaskStartScheduler inicializa a o gerenciador de tarefas (escalonador de processos), sendo, que a partir deste momento, a tarefa entra na fila para ser executada pelo FreeRTOS.

Outro fator que devemos notar quando usamos o FreeRTOS é o uso da função vTaskDelay que, ao contrário de funções como delay ou delay_ms, não é bloqueante e permite que as tarefas continuem em execução durante o seu uso.

No programa apresentado a seguir, imagine um cenário onde um LED precisa alternar o seu estado a cada 1.000ms e outro LED a cada 750ms, sendo que isso deverá ocorrer simultaneamente. Então, serão criadas duas tarefas (vTarefa1 e vTarefa2) e o FreeRTOS utilizará seu gerenciador de tarefas (escalonador de processos) para executá-las de forma concorrente no microcontrolador.

```
#include <Arduino_FreeRTOS.h>

void tarefa1 (void *parametros) {
  pinMode(12, OUTPUT);
  while (1) {
    Serial.println ("Tarefa LED 1");
    digitalWrite(12, HIGH);
    vTaskDelay (1000 / portTICK_PERIOD_MS);
    digitalWrite(12, LOW);
    vTaskDelay (1000 / portTICK_PERIOD_MS);
  }
}

void tarefa2 (void *parametros) {
  pinMode(11, OUTPUT);
  while (1) {
    Serial.println ("Tarefa LED 2");
    digitalWrite(11, HIGH);
    vTaskDelay (750 / portTICK_PERIOD_MS);
    digitalWrite(11, LOW);
    vTaskDelay (750 / portTICK_PERIOD_MS);
  }
}

void setup() {
  Serial.begin (9600);
  xTaskCreate (tarefa1, "Led1", 128, NULL, 1, NULL);
  xTaskCreate (tarefa2, "Led2", 128, NULL, 1, NULL);
  vTaskStartScheduler();
}

void loop() {
}
```

No próximo programa será realizado a controle sobre a execução das tarefas, possibilitando pausar e reiniciar cada uma delas de maneira independente. Note no programa que será criada uma tarefa para o controle do LED e outra tarefa que irá monitorar a entrada de dados através da porta serial. Quando for digitado no monitor serial as letras ‘s’ ou ‘r’ a tarefa que controla o LED será suspensa ou resumida, isto é, irá continuar do ponto que havia sido pausada.

```
#include <Arduino_FreeRTOS.h>
```

```
TaskHandle_t hTarefaLed;
TaskHandle_t hTarefaSerial;

void tarefaLed (void *parametros) {
  pinMode(12, OUTPUT);
  while (1) {
    digitalWrite(12, HIGH);
    vTaskDelay (500 / portTICK_PERIOD_MS);
    digitalWrite(12, LOW);
    vTaskDelay (500 / portTICK_PERIOD_MS);
  }
}

void tarefaSerial(void* pvParameters) {
  Serial.begin(9600);
  while (1) {
    if (Serial.available() > 0) {
      switch(Serial.read()) {
        case 's':
          vTaskSuspend(hTarefaLed);
          Serial.println("Suspensa!");
          break;
        case 'r':
          vTaskResume(hTarefaLed);
          Serial.println("Resumida!");
          break;
      }
      vTaskDelay(10 / portTICK_PERIOD_MS);
    }
  }
}

void setup() {
  xTaskCreate(tarefaLed, "Led", 128,  NULL, 1, &hTarefaLed);
  xTaskCreate(tarefaSerial, "Serial", 128, NULL, 1, &hTarefaSerial);
  vTaskStartScheduler();
}

void loop() {
}
```

Semáforos

Semáforos são usados para evitar o acesso concorrente à determinado recurso e consistem em um mecanismo que permite simplificar a exclusão mútua e a sincronização condicional entre processos, sendo um dos principais mecanismos usados em projetos de sistemas operacionais. Os semáforos podem ser classificados como:

- Binários, também chamados de mutexes (mutual exclusion semaphores), podem assumir apenas valores binários, ou seja, zero ou um.

- Contadores podem assumir qualquer valor inteiro positivo, além do zero.

No programa apresentado a seguir, o semáforo do tipo mutex será usado para evitar que os LEDs conectados aos pinos 11 e 12 acendam simultaneamente.

```
#include <Arduino_FreeRTOS.h>
#include <semphr.h>

SemaphoreHandle_t semaforoLed;

void tarefa1(void *parametros) {
  pinMode(12, OUTPUT);
  while (true) {
    Serial.print("Tarefa 1 - ");
    if (xSemaphoreTake(semaforoLed, (TickType_t) 5) == pdTRUE) {
      Serial.println("LED 1: Ligado");
      digitalWrite(12, HIGH);
      vTaskDelay(1250 / portTICK_PERIOD_MS);
      digitalWrite(12, LOW);
      xSemaphoreGive(semaforoLed);
      vTaskDelay(1250 / portTICK_PERIOD_MS);
    }
    else
      Serial.println("1 Aguardando...");
  }
}

void tarefa2(void *parametros) {
  pinMode(11, OUTPUT);
  while (true) {
    Serial.print("Tarefa 2 - ");
    if (xSemaphoreTake(semaforoLed, (TickType_t) 5) == pdTRUE) {
      Serial.println("LED 2: Ligado");
      digitalWrite(11, HIGH);
      vTaskDelay(500 / portTICK_PERIOD_MS);
      digitalWrite(11, LOW);
      xSemaphoreGive(semaforoLed);
      vTaskDelay(500 / portTICK_PERIOD_MS);
    }
    else
      Serial.println("2 Aguardando...");
  }
}
```

```
void setup() {
  Serial.begin(9600);
  semaforoLed = xSemaphoreCreateMutex();
  xSemaphoreGive(semaforoLed);
  xTaskCreate(tarefa1, "Led1", 128, NULL, 1, NULL);
  xTaskCreate(tarefa2, "Led2", 128, NULL, 1, NULL);
  vTaskStartScheduler();
}

void loop() {
}
```

Ao acessar o monitor serial é possível observar o funcionamento do semáforo. Note que enquanto um processo está executando a sua região crítica, que neste exemplo consiste em acender o LED aguardar um tempo e apagá-lo em seguida, o outro processo não consegue entrar na sua região crítica, conforme mostra a Figura 3. Com intuito de facilitar o entendimento do conceito de semáforos, vamos definir que região crítica consiste na parte de diferentes processos que nunca podem ser executadas simultaneamente.

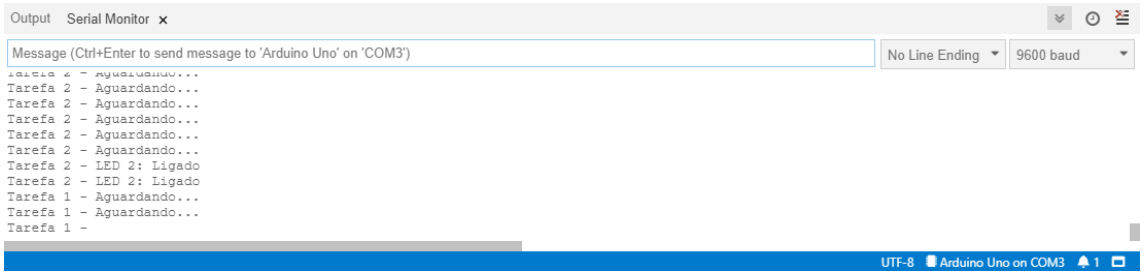


Figura 3: Semáforo

No trecho de programa apresentado a seguir podemos ver a região crítica do processo, sendo que ela apenas é executada caso o semáforo possa ser alocado para o respectivo processo, isto é, a função xSemaphoreTake retorna o valor verdadeiro. Desta forma, os demais processos que usam o mesmo semáforo, não irão conseguir executar sua região crítica enquanto o semáforo não for liberado (xSemaphoreGive) e devidamente alocado pelo processo.

```
if (xSemaphoreTake(semaforoLed, (TickType_t) 5) == pdTRUE) {
  Serial.println("LED 2: Ligado");
  digitalWrite(11, HIGH);
  vTaskDelay(500 / portTICK_PERIOD_MS);
  digitalWrite(11, LOW);
  xSemaphoreGive(semaforoLed);
  vTaskDelay(500 / portTICK_PERIOD_MS);
}
```